Solution Architecture

Phase 3: Project Design Phase

1. Architectural Goals

The architecture separates user interaction, request orchestration, AI generation and persistence. This makes the application easier to understand and maintain because each layer has a defined responsibility. The frontend does not need direct access to the Groq key or privileged database credentials.

2. Frontend Layer

The React and TypeScript frontend collects trip preferences, sends API requests and renders the response. It is responsible for presenting generation status, displaying the itinerary, showing saved trips and opening external search links. Frontend state should not be treated as the authoritative copy of a saved trip; the backend and database provide persistence.

3. API Layer

FastAPI provides routes for health status, itinerary generation, saved-trip list/retrieve/delete operations and hotel/restaurant search URLs. The API validates inputs and coordinates calls to the AI service and database client. Errors should be returned as appropriate HTTP responses so the interface can communicate failure clearly.

4. AI Service Layer

The AI service builds a prompt from user preferences and calls Groq. The response is parsed and checked for expected itinerary fields. This service boundary allows model selection, prompt changes and output validation to evolve without placing all AI-specific code in the route handlers.

5. Persistence Layer

Supabase stores the trip and itinerary records. The backend inserts records after successful generation and provides routes to list, retrieve and delete them. Database operations should handle missing records and service errors without leaving confusing partial state.

6. External Search Flow

Hotel and restaurant endpoints construct Google Maps query URLs. No live inventory database is queried by these endpoints. This keeps the integration simple but means that listing details, opening hours, availability and dietary claims are controlled by external search results and must be checked independently.

7. Operational and Security Notes

- Store API keys in backend environment configuration.
- Use CORS settings appropriate to the deployed frontend.
- Add request timeouts and controlled retries around provider calls.
- Review Supabase permissions and user ownership before public multi-user use.
- Add logs for errors while excluding secrets and unnecessary personal data.
- Monitor model quotas and hosting cold starts.

8. Architecture Limitations

The current architecture is appropriate for a prototype, but additional controls are needed for a production service. Authentication, rate limiting, automated tests, observability and robust data ownership should be added before exposing the application to a large audience.